

HELLOAGENTS • CHAPTER 8.4

构建智能文档问答助手

把 *RAG 检索* 与 *多层记忆* 接成一条学习闭环

主讲人：柠檬酸

PDF 很多，真正的问题不是“打不开”

传统阅读方式难以快速定位信息，也难以把一次问答沉淀成持续生长的知识。

从“翻文档”
升级为
“带记忆地学习”

01 / LOCATE

关键信息定位慢

02 / CONNECT

知识之间缺少关联

03 / REMEMBER

问完即忘，无法积累

一个助手，同时承担四种任务

以 Happy-LLM PDF 为例，构建基于 Gradio、RAGTool 与 MemoryTool 的交互式学习助手。

01

智能文档处理

PDF 转 Markdown、结构化分块、向量索引。

MarkItDown → Chunk → Embed

02

高级检索问答

通过 MQE 与 HyDE 改善召回和语义匹配。

MQE + HyDE + Context

03

多层次记忆

工作、情景、语义、感知记忆协同记录。

Working / Episodic / Semantic

04

个性化学习

基于历史进行回顾、统计、推荐与报告。

Recall → Stats → Report

五个步骤，形成一条完整学习闭环

原书图 8.6 的五步主链：文档与检索结果持续写入记忆，最终汇总为学习报告。



PDF 文档处理 → RAG 检索问答 → 记忆系统 → 集成助手 → 学习报告

Web 应用由三个部分拼成

课堂讲解将重点放在核心助手类和业务方法；Gradio 界面作为承载层交代。



先给每个用户一套独立的学习空间

核心类同时建立用户身份、会话、两类工具和统计容器。

```
# 为用户建立独立的记忆与知识库空间
def __init__(self, user_id="default_user"):
    self.user_id = user_id
    self.session_id = f"session_{datetime.now():%Y%m%d_%H%M%S}"
    self.memory_tool = MemoryTool(user_id=user_id)
    self.rag_tool = RAGTool(rag_namespace=f"pdf_{user_id}")
    self.stats = {"session_start": datetime.now(), ...}
    self.current_document = None
```

user_id

隔离不同用户的四类记忆。

rag_namespace

隔离每个用户的 PDF 知识库。

session_id

串起一次学习会话的完整过程。

完整初始化代码与统计字段

四个字段， 分别解决四个边界问题

MEMORY 用户隔离 不同用户拥有独立的记忆空间。 user_id	RAG 知识库隔离 PDF 索引不会在用户间串线。 pdf_{user_id}	SESSION 会话追踪 一次学习过程可被完整回顾。 session_YYYYMMDD	STATS 指标累计 为学习报告预先保存关键计数。 documents / questions / concepts
--	---	---	--

加载 PDF，不只是“放进向量库”

处理成功后还要更新当前文档、累计统计，并把“加载过什么”写入情景记忆。

```
# Markdown 转换 → 智能分块 → 向量化
result = self.rag_tool.execute(
    "add_document",
    file_path=pdf_path,
    chunk_size=1000,
    chunk_overlap=200
)
if result.get("success", False):
    self.current_document = os.path.basename(pdf_path)
    self.stats["documents_loaded"] += 1
```

INPUT

校验 PDF 路径。

RAG

转换、增强、分块、向量化。

MEMORY

记录文档加载事件。

含错误处理、计时与记忆写入

一行调用，封装完整文档处理链

OPERATION	SOURCE	CHUNK	RESULT
add_document 声明本次是添加文档操作。	file_path 指向待处理 PDF 文件。	1000 / 200 控制文本块大小与重叠范围。	状态字典 返回成功状态、错误或统计信息。
execute("add_document")	Happy-LLM-0727.pdf	chunk_size / overlap	success / message / stats

为什么“加载文档”属于情景记忆？

EVENT	VALUE
它是发生过的一件事 有明确时间、对象与会话上下文：用户在本次学习中加载了某份文档。 memory_type = "episodic"	它让助手能回顾历程 以后可以回答“我之前加载过哪些文档？”，也能追踪文档使用情况。 event_type = "document_loaded"

一次提问，会同时经过检索和记忆

问题先进入工作记忆，RAG 负责寻找答案，问答事件再沉淀到情景记忆。



QUESTION → WORKING MEMORY → RAG → EPISODIC MEMORY → ANSWER

工作记忆管“现在”，情景记忆管“发生过”

WORKING MEMORY 当前问题与上下文 服务这一次回答，让系统知道眼下正在处理什么。 <code>content = f"提问: {question}"</code>	EPISODIC MEMORY 一次问答事件 服务未来回顾，让系统知道用户曾围绕什么主题学习。 <code>event_type = "qa_interaction"</code>
--	---

同一个问题，换几个角度再去找

多查询扩展生成语义等价但表述不同的查询，从多个角度理解用户意图。

ORIGINAL

什么是大语言模型？

用户的原始提问。

`question`

EXPANSION

生成多种说法

定义是什么？请解释一下？LLM 是什么意思？

`_generate_multi_queries()`

RECALL

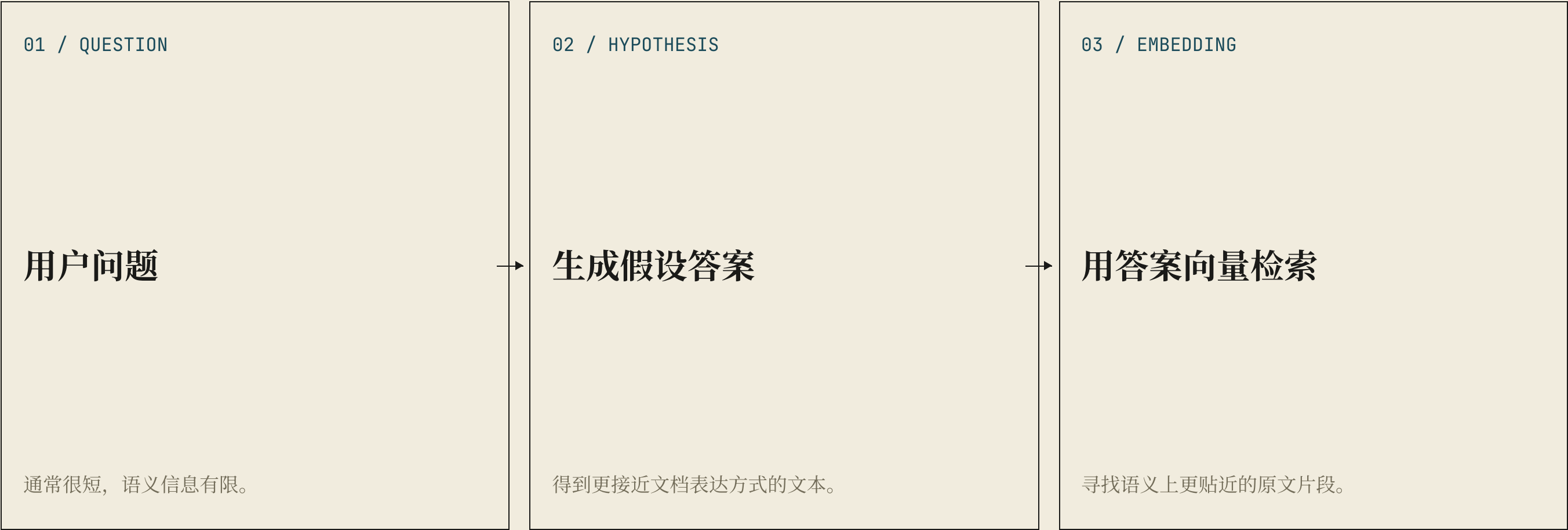
扩大召回面

用多个查询覆盖不同措辞的相关片段。

`multi-angle retrieval`

先写一份“可能的答案”，再拿它去检索

假设文档嵌入用答案式文本桥接短查询与原文段落之间的语义鸿沟。



SHORT QUERY → HYPOTHETICAL DOCUMENT → DOCUMENT-LIKE EMBEDDING

问答之外，还要把学习过程变成资产

add_note() 记录笔记 把理解过的概念保存进语义记忆。 knowledge	recall() 回顾历程 从记忆系统中搜索过去的学习内容。 history	get_stats() 查看统计 计算会话时长和各类活动数量。 metrics	generate_report() 生成报告 汇总记忆、RAG 状态与学习指标。 report.json
---	--	---	--

笔记记录的是“我理解了什么”

```
# 概念性理解写入语义记忆
def add_note(self, content, concept=None):
    self.memory_tool.execute(
        "add",
        content=content,
        memory_type="semantic",
        importance=0.8,
        concept=concept or "general"
    )
```

SEMANTIC

保存概念性知识，而不是一次事件。

CONCEPT

用概念标签帮助后续组织与搜索。

STATS

每新增一条笔记，已学概念数加一。

保留 session_id 与计数更新

一边回顾内容，一边观察学习进度

RECALL 按主题搜索学习历程 直接调用 MemoryTool 的 search，根据查询返回相关记忆。 search(query, limit=5)	STATS 把本次学习量化 会话时长、加载文档、提问次数、学习笔记与当前文档。 duration + counters + current_document
---	---

最后，把一次学习会话打包成报告

报告合并会话信息、学习指标、记忆摘要与 RAG 状态，并可保存为 JSON 文件。

session_info 这次学习是谁、何时、多久 记录 session_id、user_id、开始时间与会话持续秒数。 identity + timeline	learning_metrics 这次学习做了多少事 汇总加载文档、提问次数和学习概念数量。 documents + questions + concepts	memory_summary / rag_status 记住了什么，知识库如何 通过两类工具分别取得记忆摘要和 RAG 统计。 summary(limit=10) + stats()
--	---	---